Recap

1. What are the different types of arguments?
2. What is default value argument?
3. What is the benefit of having keyword arguments?

```
# def name(arguments...)

def intro(name, age, grade):
  print(f"hello {name}, you are {age} years old and {grade} grade")

intro(age=22, name="nani", grade=11)
# intro(22, "nani", 11)
```

```
hello nani, you are 22 years old and 11 grade
```

What is the output of the following function?

```
def myFunction(name="Addis", height):
  print(f"hello I am {name} and I am {height} tall.")

myFunction(23)

#check the code here
def myFunction(name="Addis", height):
  print(f"hello I am {name} and I am {height} tall.")

# myFunction(23)
```

```
  File "/tmp/ipykernel_6646/2573489963.py", line 2
    def myFunction(name="Addis", height):
                                        ^
SyntaxError: parameter without a default follows parameter with a
default
```

How about this

```
def myFunction(height, name="Addis"):
  print(f"hello I am {name} and I am {height} tall.")

myFunction(23)

#check code here
def myFunction(height, name="Addis"):
  print(f"hello I am {name} and I am {height} tall.")

myFunction(23)
```

```
hello I am Addis and I am 23 tall.
```

What's the output for the following

```
def sum(greeting, *arguments):
  print(f"Hello {greeting}")
  sum = 0
  for value in arguments:
    sum += value

  return sum

sum("How are you", 20, 12, -9, 100)

#check code here
def sum(greeting, *arguments):
  print(f"Hello {greeting}")
  sum = 0
  for value in arguments:
    sum += value

  return sum

sum("How are you", 20, 12, -9, 100)

Hello How are you

123
```

Introduction to Object Oriented Programming

1. Understanding Procedural Programming

Procedural programming is a programming paradigm that focuses on writing code as a sequence of procedures (functions) that perform operations on data.

Key Characteristics:

- *Data and functions are separate*
- *Code is executed step-by-step*
- *Uses loops, conditionals, and functions*

Let's look at an Example: `Bank Account Management (Procedural Approach)`

```
# Data (Variables)
account1_name = "Alemu"
account1_balance = 1000
```

```
account2_name = "Birtikuan"
account2_balance = 2000

# Functions (Procedures)
def deposit(name, balance, amount):
    print(f"Depositing ${amount} to {name}'s account")
    return balance + amount

def withdraw(name, balance, amount):
    print(f"Withdrawing ${amount} from {name}'s account")
    if amount > balance:
        print("Insufficient funds!")
        return balance
    return balance - amount

# Operations
account1_balance = deposit(account1_name, account1_balance, 500)
account2_balance = withdraw(account2_name, account2_balance, 300)
account1_balance = withdraw(account1_name, account1_balance, 2000)  #
this Should fail
```

```
Depositing $500 to Alemu's account
Withdrawing $300 from Birtikuan's account
Withdrawing $2000 from Alemu's account
Insufficient funds!
```

```
# Display final balances
print("\nFinal Balances:")
print(f"{account1_name}: ${account1_balance}")
print(f"{account2_name}: ${account2_balance}")
```

```
Final Balances:
Alemu: $1500
Birtikuan: $1700
```

What's wrong with procedural approach?

- **Data and functions are separate → Harder to manage**
- **Many variables needed → More error-prone**
- **Difficult to scale → Adding new features becomes messy**

2. Introducing Object Oriented Programming(OOP)?

OOP is a programming paradigm that bundles **data (attributes)** and **functions (methods)** **`into objects.`**

Key Concepts:

- **Class**: `A blueprint for creating objects` (e.g., BankAccount)
- **Object**: `An instance of a class` (e.g., account1 = BankAccount("Alemu", 1000))

- **Attributes**: `Variables that belong to an object` (e.g., name, balance)
- **Methods**: `Functions that belong to an object` (e.g., deposit(), withdraw()

Bank account:

- attributes: name, account number, balance, date of birth, Id number
- methods: withdraw, deposit, ask_for_loan, transfer

Animal:

- attributes: color, number_of_legs, noise...
- methods: move, sleep, eat...

Student:

- attributes: name, grade, age, school
- methods: apply_to_universities, book_library,

`Let's convert the previous example Bank Account Management to OOP Approach`

```
class BankAccount:
    def __init__(self, name, accountNumber, balance=0): #constructor
        self.name = name  # Attribute
        self.balance = balance  # Attribute
        self.accountNumber = accountNumber

    def deposit(self, amount):  # Method
        print(f"Depositing ${amount} to {self.name}'s account")
        self.balance += amount

    def withdraw(self, amount):  # Method
        print(f"Withdrawing ${amount} from {self.name}'s account")
        if amount > self.balance:
            print("Insufficient funds!")
            return
        self.balance -= amount

    def display_balance(self):  # Method
        print(f"{self.name}'s balance: ${self.balance}")

# Creating objects
account1 = BankAccount("Alemu", 1000, 4000)
account2 = BankAccount("Birtikuan", 2000)
account3 = BankAccount("Abebe", 10000)
william = BankAccount("William", 123456, 40000)
william.display_balance()
# Performing operations
# account1.display_balance()
# account1.deposit(1000)
# account1.display_balance()
```

```
# account1.deposit(500)
account2.display_balance()
account2.deposit(500)
account2.withdraw(300)
account2.display_balance()
# account1.withdraw(2000)  # Should fail

# # Displaying final balances
# print("\nFinal Balances:")
# account1.display_balance()
# account2.display_balance()
```

```
William's balance: $40000
Birtikuan's balance: $0
Depositing $500 to Birtikuan's account
Withdrawing $300 from Birtikuan's account
Birtikuan's balance: $200
```

```
account1.name, account2.name
```

```
('Alemu', 'Birtikuan')
```

Advantages of OOP:

- ***Data and functions are bundled together → Easier to manage***
- ***Fewer variables to track → Less error-prone***
- ***More organized and scalable → Adding new features is cleaner***

```
# add fayda id number
class BankAccount:
    def __init__(self, name, accountNumber, fayda_id, balance=0):
#constructor
        self.name = name  # Attribute
        self.balance = balance  # Attribute
        self.accountNumber = accountNumber
        self.fayda_id = fayda_id

    def deposit(self, amount):  # Method
        print(f"Depositing ${amount} to {self.name}'s account")
        self.balance += amount

    def withdraw(self, amount):  # Method
        print(f"Withdrawing ${amount} from {self.name}'s account")
        if amount > self.balance:
            print("Insufficient funds!")
            return
        self.balance -= amount

    def display_balance(self):  # Method
        print(f"{self.name}'s with {self.fayda_id} id balance is: $
```

```
{self.balance}")

# Creating objects
account1 = BankAccount("Alemu", 1000, "et3456", 4000)
account2 = BankAccount("Birtikuan", 2000, "et3436")
account3 = BankAccount("Abebe", 10000, "et3476")
william = BankAccount("William", 123456, "et3496", 40000)
william.display_balance()
# Performing operations
# account1.display_balance()
# account1.deposit(1000)
# account1.display_balance()
# account1.deposit(500)
account2.display_balance()
account2.deposit(500)
account2.withdraw(300)
account2.display_balance()
# account1.withdraw(2000)  # Should fail

# # Displaying final balances
# print("\nFinal Balances:")
# account1.display_balance()
# account2.display_balance()
```

```
William's with et3496 id balance is: $40000
Birtikuan's with et3436 id balance is: $0
Depositing $500 to Birtikuan's account
Withdrawing $300 from Birtikuan's account
Birtikuan's with et3436 id balance is: $200
```

3. Key Differences: Procedural vs. OOP

- **Structure**:
  - `Procedural`: Functions + Data (separate)
  - `OOP`: Objects (data + methods)
- **Approach**:
  - `Procedural`: Step-by-step execution
  - `OOP`: Modeling real-world entities
- **Scalability**:
  - `Procedural`: Harder to maintain
  - `OOP`: Easier to extend

Example:

- Procedural:

```
deposit(name, balance, amount)
```

- OOP:

```
account1.deposit(amount)
```

4. Classes and Objects

Class:

- A **class** is a ***`blueprint/template`*** for creating objects.
- Defines `attributes` (data) and `methods` (functions) ***`that the objects will have.`***

Object:

- An **object** is an ***`instance of a class`***.
- ***`Each object has its own separate data`***.

Analogy:

- **Class** = *`Car`* (defines what a car is in general)
- **Object** = *`Nissan Patrol(an actual car)`* (instance created from the Car class)

Class Structure in Python

Basic syntax:

```
class ClassName:
    def __init__(self, param1, param2):  # Constructor
        self.attribute1 = param1  # Instance attribute
        self.attribute2 = param2

    def method1(self):  # Instance method
        # Method body
```

Key Components:

- `__init__ method`: **Constructor** (runs when object is created)
- `self`: **Refers to the current object instance**
- `Attributes`: **Variables belonging to the object** (self.attribute)
- `Methods`: **Functions belonging to the object**

`Let's look at Car class example in detail:`

```
class Car:
    # Class attribute (shared by all instances)
    wheels = 4
```

```
    def __init__(self, brand, model, color):
        # Instance attributes
        self.brand = brand
        self.model = model
        self.color = color
        self.speed = 0  # Default value

    def accelerate(self, increase):
        self.speed += increase
        print(f"Accelerating! Speed is now {self.speed} km/h")

    def brake(self, decrease):
        if self.speed - decrease < 0:
            self.speed = 0
        else:
            self.speed -= decrease
        print(f"Braking! Speed is now {self.speed} km/h")

    def display_info(self):
        print(f"{self.color} {self.brand} {self.model} | Speed:
{self.speed} km/h | Wheels: {self.wheels}")
```

Creating objects

```
car1 = Car("Toyota", "Corolla", "Red")
car2 = Car("Tesla", "Model 3", "Blue")

# Using objects
car1.accelerate(30) #speed = 30
car2.accelerate(50) #speed  = 50
car1.brake(10) # speed = 20

print("\nCar Details:")
car1.display_info()
car2.display_info()
```

```
Accelerating! Speed is now 30 km/h
Accelerating! Speed is now 50 km/h
Braking! Speed is now 20 km/h

Car Details:
Red Toyota Corolla | Speed: 20 km/h | Wheels: 4
Blue Tesla Model 3 | Speed: 50 km/h | Wheels: 4
```

```
class Dog:
    # Class attribute (shared by all instances)
    species = "Canis familiaris"

    # Initializer (constructor)
    def __init__(self, name, age):
```

```
        # Instance attributes
        self.name = name
        self.age = age

    # Instance method
    def bark(self):
        print(f"{self.name} says woof!")

    def describe(self):
        print(f"{self.name} is {self.age} years old and is a
{self.species}")

# Creating objects
dog1 = Dog("Fido", 3)
dog2 = Dog("Buddy", 5)

# Using objects
dog1.bark()
dog2.describe()

# Accessing attributes
print(f"{dog1.name} is a {dog1.species}")
print(f"{dog2.name} is {dog2.age} years old")
```

```
Fido says woof!
Buddy is 5 years old and is a Canis familiaris
Fido is a Canis familiaris
Buddy is 5 years old
```

Understanding the self Parameter

- Refers to the **current object instance**
- **`Python automatically passes it when calling methods`**

Example:

`The following two are equivalent`

```
car1.accelerate(30)
Car.accelerate(car1, 30)  # Rarely used this way

# Demonstrating what 'self' refers to
class SelfDemo:
    def show_self(self):
        print(f"Inside the method, self is: {self}")
        print(f"Memory address: {id(self)}")

demo = SelfDemo()
print(f"The object is: {demo}")
print(f"Memory address: {id(demo)}")
```

```
print("Calling method...")
demo.show_self()
```

```
The object is: <__main__.SelfDemo object at 0x7fc0a1e68200>
Memory address: 140465326686720
Calling method...
Inside the method, self is: <__main__.SelfDemo object at
0x7fc0a1e68200>
Memory address: 140465326686720
```

Adding Attributes Dynamically

```
car1.vin = "ABC123"  # Add new attribute to car1 only
print(f"VIN: {car1.vin}")  # Works
# print(f"VIN: {car2.vin}")  # Would cause AttributeError
```

```
VIN: ABC123
```

Common Mistakes & Fixes

Mistake 1: Forgetting self

```
class BuggyCar:
    def __init__(brand, model):  # Missing self!
        brand = brand  # Wrong!
        model = model

    def drive():  # Missing self!
        print("Driving")
```

`Fixed version`

```
class CorrectCar:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def drive(self):
        print(f"{self.model} is driving")
```

Mistake 2: Confusing Class/Instance Attributes

```
class Counter:
    count = 0  # Class attribute

    def increment(self):
        self.count += 1  # Creates instance attribute!

# Solution:
    def increment(self):
        Counter.count += 1  # Explicitly modify class attribute
```

```
class Bird:
  legs = 2

  def __init__(self, color, species):
    self.color = color
    self.species = species

  def fly(self):
    print(f"The {self.color} bird is flying!")


#object of our bird class
bird1 = Bird("Grey", "Crow")
bird2 = Bird("Blue", "Pigeon")

bird1.fly()
bird2.fly()
print(bird1.legs)
print(bird2.legs)
```

```
The Grey bird is flying!
The Blue bird is flying!
2
2
```

```
bird1.legs = 4
print(bird1.legs)
```

```
4
```

```
bird2.legs
```

```
2
```

```
Bird.legs = 6
bird1.legs, bird2.legs
```

```
(4, 6)
```

5. Let's make hospital management system

What are the classes we need?

- Patient
- Doctor
- Nurse
- Hospital

What are the attributes for each class?

What are the behaviros(methods) for each class?

```
#write your code here for the first class
#Patient Attributes
'''
Attributes: name, age, gender, nationality, card number..
methods: book_appointment, take-medicine, register...
'''

#Doctor attributes
'''
attributes: NAME, SPECIALITY, GENDER, DEPARTMENT..
METHODS: PERSCRIBE, CHECKIN, CHECKOUT..
'''

#Write your code here for the second class

#write your code here for the third class

#write your code here for the fourth class
```

6. Let's practice

I. Exercise 1: Convert Procedural Code to OOP.

`Let's take the below procedural code and rewrite it using OOP:` It's about Managing a car's fuel level

```
car_fuel = 50
car_model = "Toyota"

def add_fuel(fuel, amount):
    print(f"Adding {amount}L of fuel")
    return fuel + amount

def drive(fuel, distance):
    fuel_needed = distance * 0.1
    if fuel_needed > fuel:
        print("Not enough fuel!")
        return fuel
    print(f"Driving {distance}km")
    return fuel - fuel_needed

# Operations
car_fuel = add_fuel(car_fuel, 20)
car_fuel = drive(car_fuel, 100)
car_fuel = drive(car_fuel, 500)  # Should fail
```

`Rewrite this using a Car class with add_fuel() and drive() methods.`

II. Exercise 2: Let's Create a Student Class.

Define a Student class with:

- **Attributes**: `name, age, grade`
- **Methods**:
  - `promote()` → Increases grade by 1
  - `display_info()` → Prints student details

Example of the usage:

```
student1 = Student("Alice", 15, 9)
student1.promote()
student1.display_info()  # Output: "Alice is 15 years old and in grade 10"
```

V. Exercise 4: Let's create Rectangle Class

Create a Rectangle class with:

- **Attributes**: `width, height`
- **Methods**: `area(), perimeter(), is_square()`

Example usage:

```
rect = Rectangle(5, 5)
print(rect.area())       # 25
print(rect.perimeter())  # 20
print(rect.is_square())  # True
```

IV. Exercise 4: Let's Fix the Code

`This code has errors. Identify and fix them:`

```
class Dog:
    def __init__(self, name, age):
        name = name
        age = age

    def bark():
        print(f"{name} says woof!")

my_dog = Dog("Buddy", 3)
my_dog.bark()
```

v. Let's create a class for a Restaurant Menu

- **Class** = `Menu`
- **Attributes**: `items, prices`
- **Methods**: `add_item(), remove_item(), display_menu()`
- **Objects** = `breakfast_menu, dinner_menu`

`Note: Each Menu has its own items and prices.`

```
'''
in the custom loop for finding the minimum value ,we initalize the
minimum
 variable with the first element of the list (l[0]). what happens if
the list we modify the function is empty ? how would
 we modify the function to handle an empty list without the code
crashing ?
'''

l = [23, 90, 90. -9, -45, -13, 78]
minimum = l[0]
for val in l:
  if val < minimum:
    minimum = val
print(minimum)
```

```
-45
```

```
def find_minimum(l):
  minimum = l[0]
  for val in l:
    if val < minimum:
      minimum = val
  print(minimum)

l2 = [89, 65, -90, 23]
find_minimum(l2)
```

```
-90
```

```
l3=[]
find_minimum(l3)
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
/tmp/ipykernel_6646/1605566091.py in <cell line: 0>()
      1 l3=[]
```

```
----> 2 find_minimum(l3)

/tmp/ipykernel_6646/3867020671.py in find_minimum(l)
      1 def find_minimum(l):
----> 2   minimum = l[0]
      3   for val in l:
      4     if val < minimum:
      5       minimum = val

IndexError: list index out of range
```

```python
def find_minimum(l):
  if len(l) == 0:
    return "Your list is empty!"
  minimum = l[0]
  for val in l:
    if val < minimum:
      minimum = val
  print(minimum)

find_minimum(l3)
```

{"type":"string"}